

```
+++ date = '2026-04-30T01:04:23-07:00' draft = false title = 'Practica2: Programacion Orientada Objetos'
```

```
+++
```

Universidad Autonoma de Baja California

Materia: Paradigmas de la programacion

Docente: Jose Carlos Gallegos Mariscal

Alumno: Carlos David Cruz Santiesteban

Matricula: 379909

Practica 2: Programacion Orientada a Objetos (simulación de sistema de estacionamiento)

Introducción:

En esta práctica desarrollamos una simulación de un sistema de estacionamiento aplicando conocimientos del paradigma de Programacion Orientado a Objetos. Es un sistema sencillo en el que permite marcar entradas y salidas de vehiculos, calcular el cobro segun el tiempo que hayan estado y consultar cual es el estado del estacionamiento.

Modelo del dominio

Clases y sus responsabilidades

Clases	Responsabilidad
Vehicle	Clase base para vehiculos, tiene placa y tipo (carro o moto)
Car	Herada de Vehicle
Motorcycle	Herada de Vehicle
ParkingSpot	Representa un lugar fisico del estacionamiento
Ticket	Registra la estancia de un vehiculo
RatePolicy	Clase base para las politicas de cobro
HourlyRatePolicy	Calcula costo por hora segun tipo de vehiculo
FlatRatePolicy	Calcula un costo fijo sin importar horas ni tipo
ParkingLot	Administra spots, tickets y la politica de cobro

Evidencia de conceptos

Encapsulación

En el siguiente fragmento de codigo, podemos observar que los atributos de las clases son privados (utilizamos doble guion bajo '_' para marcar que son privados) y que su modificacion solo es con metodos,

```
# Fragmento de codigo de Spot.py
class ParkingSpot:
    def __init__(self, spot_id, allowed_type):
        self.__spot_id = spot_id
        self.__allowed_type = allowed_type # "Car", "Motorcycle" o "Any"
        self.__occupied = False
        self.__current_vehicle = None

    def park(self, vehicle):
        if self.__occupied:
            print(f" Error: el spot {self.__spot_id} ya esta ocupado.")
            return False
        self.__current_vehicle = vehicle
        self.__occupied = True
        return True
```

Abstracción

El cobro esta separado del resto del sistema, mediante la clase RatePolicy. ParkingLot no hace el calculo del costo, llama a la funcion calculate

```
# Fragmento de codigo de RatePolicy.py
class RatePolicy:
    def calculate(self, hours, vehicle):
        if vehicle.get_type() == "Car":
            return hours * self.__car_rate
        else:
            return hours * self.__moto_rate
```

```
# Fragmento de codigo de ParkingLot.py
def exit(self, ticket_id, hours):
    # Buscar el ticket activo
    ticket = None
    for t in self.__active_tickets:
        if t.get_ticket_id() == ticket_id:
            ticket = t
            break

    if ticket is None:
        print(f" No se encontro el ticket #{ticket_id}.")
        return None

    # Calcular costo y cerrar ticket
    cost = self.__policy.calculate(hours, ticket.get_vehicle())
    ticket.close(hours, cost)
    ticket.get_spot().release()
    self.__active_tickets.remove(ticket)
```

```
self.__total_revenue += cost
return cost
```

Composición

La clase ParkingLot esta compuesto por listas de ParkingSpot, Ticket y una instancia de RatePolicy

```
# Fragmento de codigo de ParkingLot.py
class ParkingLot:
    def __init__(self, name):
        self.__name = name
        self.__spots = []          # Composicion: tiene ParkingSpots
        self.__active_tickets = [] # Composicion: tiene Tickets
        self.__policy = HourlyRatePolicy() # Composicion: tiene RatePolicy
```

Herencia

Car y Motorcycle heredan de Vehicle y especializan el tipo de vehiculo sin repetir codigo

```
# Fragmento de codigo de Vehicle.py
class Vehicle:
    def __init__(self, plate, vehicle_type):
        self.__plate = plate
        self.__type = vehicle_type

    def get_plate(self):
        return self.__plate

    def get_type(self):
        return self.__type

    def get_type(self):
        return self.__type

    def __str__(self):
        return f"{self.__type} - {self.__plate}"

class Car(Vehicle):
    def __init__(self, plate):
        super().__init__(plate, "Car")

class Motorcycle(Vehicle):
    def __init__(self, plate):
        super().__init__(plate, "Motorcycle")
```

Polimorfismo

ParkingLot usa RatePolicy sin tener una implementacion concreta. Se puede cambiar la forma de cobro (por hora o fija) en tiempo de ejecucion.

```
# Fragmento de codigo de RatePolicy.py
# Dos metodos que pueden ser usados para calculate
class HourlyRatePolicy(RatePolicy):
    def calculate(self, hours, vehicle):
        if vehicle.get_type() == "Car":
            return hours * self.__car_rate
        else:
            return hours * self.__moto_rate

class FlatRatePolicy(RatePolicy):
    def calculate(self, hours, vehicle):
        return self.__amount
```

MVC con Flask

Separación de responsabilidades

Capa	Archivo(s)	Descripción
Model	<code>models/</code>	Aqui es donde guardamos las clases (.py)
View	<code>templates/</code>	Archivos HTML: dashboard.html, entrada.html, salida.html
Controller	<code>app.py</code>	Controla el flujo de la pagina, por medio de peticiones y llamadas al modelo

Rutas implementadas

Método	Ruta	Descripción
GET	<code>/entrada</code>	Muestra el formulario de entrada
POST	<code>/entrada</code>	Procesa la entrada y registra el vehiculo
GET	<code>/salida</code>	Muestra el formulario de salida
POST	<code>/salida</code>	Procesa la salida y calcula el costo

Capturas

DashboardRegistrar EntradaRegistrar Salida

Dashboard

Ocupacion

Total: 10 | Libres: 10 | Ocupados: 0

Política actual: Por hora (Auto: \$20h | Moto: \$10h)

Total recaudado: \$0

Estado de los spots

Spot	Tipo	Estado
A1	Car	Libre
A2	Car	Libre
A3	Car	Libre
A4	Car	Libre
A5	Car	Libre
A6	Car	Libre
M1	Motorcycle	Libre
M2	Motorcycle	Libre
M3	Motorcycle	Libre
M4	Motorcycle	Libre

Tickets activos

No hay vehículos estacionados.

Registrar EntradaRegistrar Salida

DashboardRegistrar EntradaRegistrar Salida

Registrar Entrada

Placa:

avg10

Tipo de vehiculo:

Auto

Registrar Entrada

[Volver al Dashboard](#)

DashboardRegistrar EntradaRegistrar Salida

Registrar Salida

Tickets activos:

#	Vehiculo	Spot
1	Car - AZM1201	A1
2	Car - AVG10	A2

Numero de ticket:

Horas estacionado:

Registrar Salida

[Volver al Dashboard](#)

Pruebas manuales

Prueba 1:

Estado de los spots

Spot	Tipo	Estado
A1	Car	Ocupado — Car - AZM1201
A2	Car	Ocupado — Car - OBJ17
A3	Car	Libre
A4	Car	Libre
A5	Car	Libre
A6	Car	Libre
M1	Motorcycle	Ocupado — Motorcycle - AVG10
M2	Motorcycle	Libre
M3	Motorcycle	Libre
M4	Motorcycle	Libre

(Salida de obj17 despues de 3 horas)

Ocupacion

Total: 10 | Libres: 8 | Ocupados: 2

Politica actual: Por hora | Auto: \$20/h | Moto: \$10/h

Total recaudado: \$60.0

Estado de los spots

Spot	Tipo	Estado
A1	Car	Ocupado — Car - AZM1201
A2	Car	Libre
A3	Car	Libre
A4	Car	Libre
A5	Car	Libre
A6	Car	Libre
M1	Motorcycle	Ocupado — Motorcycle - AVG10

Prueba 2:

```
=====
UABC Parking
Tarifa: Por hora | Auto: $20/h | Moto: $10/h
Recaudado: $0
=====
[1] Registrar entrada
[2] Registrar salida
[3] Ver ocupacion
[4] Ver tickets activos
[5] Cambiar politica de cobro
[0] Salir
=====
Elige una opcion: 5

-- CAMBIAR POLITICA DE COBRO --
Actual: Por hora | Auto: $20/h | Moto: $10/h
[1] Por hora (Auto $20/h | Moto $10/h)
[2] Tarifa fija ($50)
Elige: 2
Politica cambiada a: tarifa fija.
```

```
=====
UABC Parking
Tarifa: Tarifa fija: $50 (sin importar horas ni tipo)
Recaudado: $0
=====
[1] Registrar entrada
[2] Registrar salida
[3] Ver ocupacion
[4] Ver tickets activos
[5] Cambiar politica de cobro
[0] Salir
=====
Elige una opcion: 2

-- REGISTRAR SALIDA --
Tickets activos:
Ticket #1 | Car - AMZ1201 | Spot A1 | ACTIVO
Numero de ticket: 1
Horas estacionado: 500
Salida registrada. Costo: $50
```

Conclusión

Con esta simulacion aplicamos los fundamentos de la programacion orientada objetos, durante la implementacion creo que aprendi bastante, y reforce varios conocimientos vistos en la propia materia de POO. El uso de la flask como herramienta, se me hizo complicado pero a la vez bastante interesante, me gustaria usarla en diferentes proyectos en un futuro.